

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Complements of Machine Learning

2025/2026



What did we see in the previous class?

- Discriminative vs. Generative models
 - **Discriminative Models:** Learn the boundary *between* classes to predict a label (e.g., "Is this a cat or a dog?").
 - **Generative Models:** Learn the underlying *distribution* of the data itself to create entirely new, realistic samples (e.g., "Draw me a completely new cat").
- Example of Generative models (mostly for images)
 - Variational auto-encoder (explicit density model)
 - GANs (implicitly density model)
- Forgot to mention:
 - SOTA for image generation are diffusion models, which we will learn in a future class.

Today's Agenda

Topic
1. How to work with text
2. RNN
3. Sequence-to-Sequence
4. Attention Mechanism

1. How to work with text

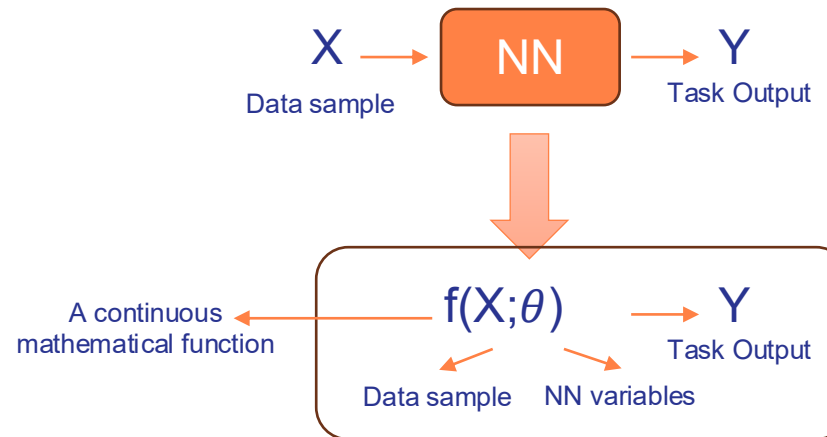
Why do we care?

- Text classification and analysis
 - Sentiment Analysis
 - Spam/Toxicity detection
- Information extraction
 - Named entity recognition
 - Relation extraction
- Text generation
 - Summarization
 - Machine translation
 - Answer generation
 - Paraphrasing
- Search and retrieval
 - Semantic search
 - Text similarity

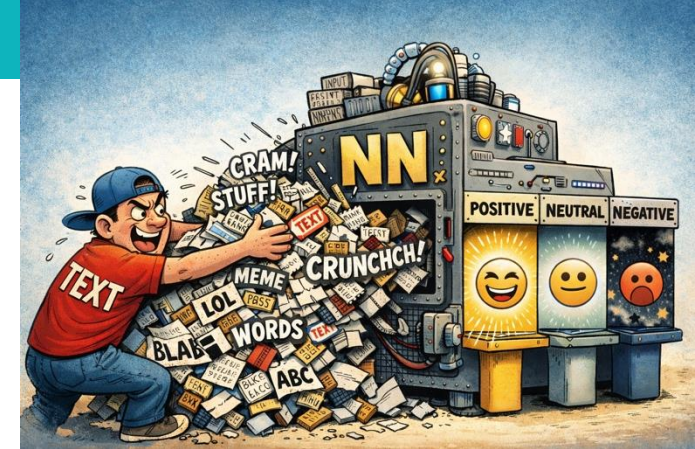
There are a lot of interesting tasks that work directly over text.

What we know so far

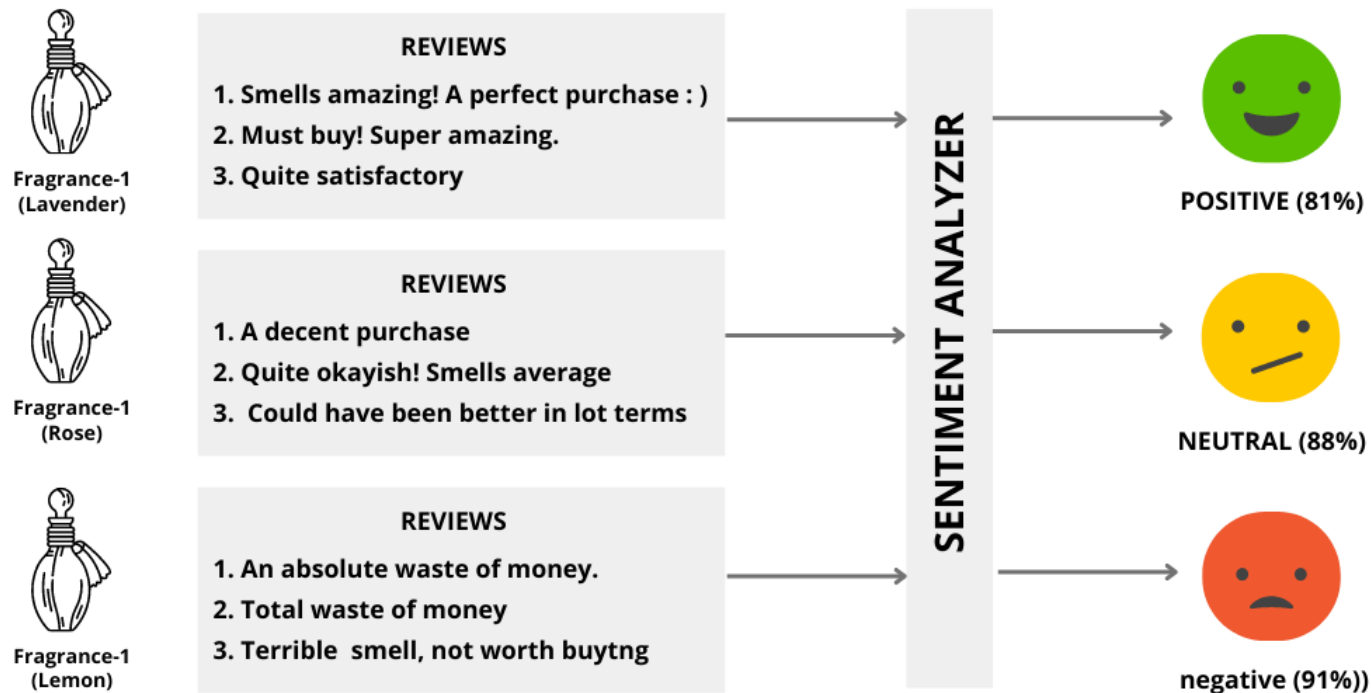
- A neural network is a mathematical continuous function.
 - Which means it only works with numbers!



How can we use it with text?

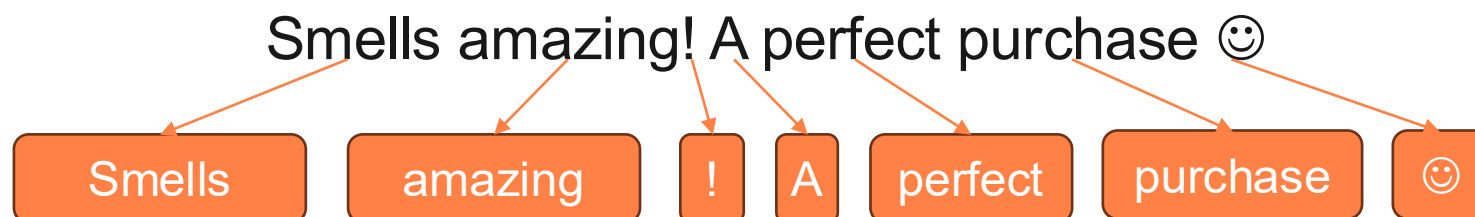


- Let's consider the following problem:
 - Given a set of human reviews, we want to get its sentiment as positive/neutral/negative.



How can we use it with text?

- Since NN only work with numbers we need to somehow encode the text into numbers.
- We can split the text into words also know as token and then convert the token themselves into some kind of numerical representation.



How to represent text?

- What is the best way to encode text?
- Word -> number?
- Dog -> 0
- Apple -> 1
- Cat -> 2

How to represent text?

➤ What is the best way to encode text?

➤ Word -> number?

➤ Dog -> 0

➤ Apple -> 1

➤ Cat -> 2

Problem:

➤ Numerical order encodes information, which is incorrect in the perspective of the words.

➤ Arithmetic does not hold.

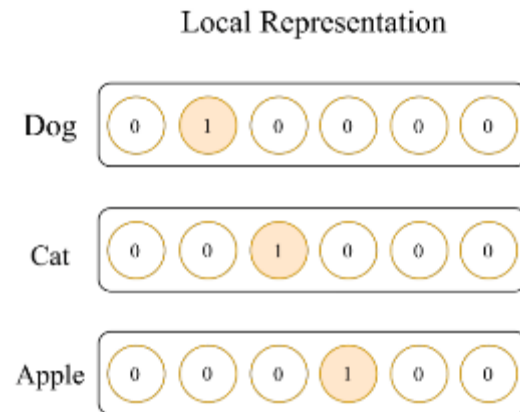
○ E.g.: if we try to do $\text{Cat-Dog} = 2-0 = 2$. So, $\text{Cat-Dog} = \text{Cat}$? It does not make sense.

➤ Relationships between words are also lost.

Let's do better!

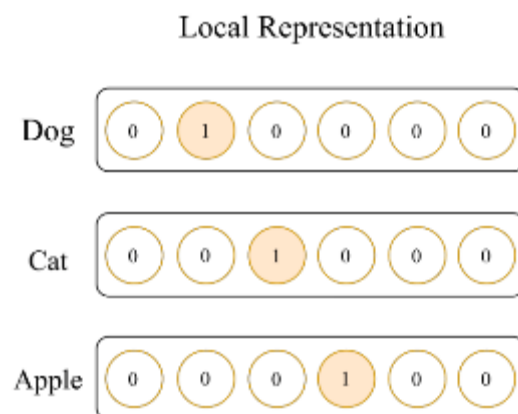
How to represent text?

- What is the best way to encode text?
- Word -> local representation (one-hot)?



How to represent text?

- What is the best way to encode text?
- Word -> local representation (one-hot)?



Problem:

- Relationships between words are still lost because the vectors are orthogonal.

Can you do better?

How to represent text?

- What is the best way to encode text?
- Word -> local representation (one-hot)?

Distributed Representation

Dog	1	0.8	0.7	0
Cat	1	0.4	0.2	0
Apple	0	0	0.3	1
	Animal	Likes running	Has color yellow	Fruit

Idea:

- Decomposing words into sets of fixed meaningful dimensions, keeping the relationship between words.

How to represent text?

- What is the best way to encode text?
- Word -> local representation (one-hot)?

Distributed Representation

Dog	1	0.8	0.7	0
Cat	1	0.4	0.2	0
Apple	0	0	0.3	1
	Animal	Likes running	Has color yellow	Fruit

Idea:

- Decomposing words into sets of fixed meaningful dimensions, keeping the relationship between words.

Next question, how can we create these representations automatically?

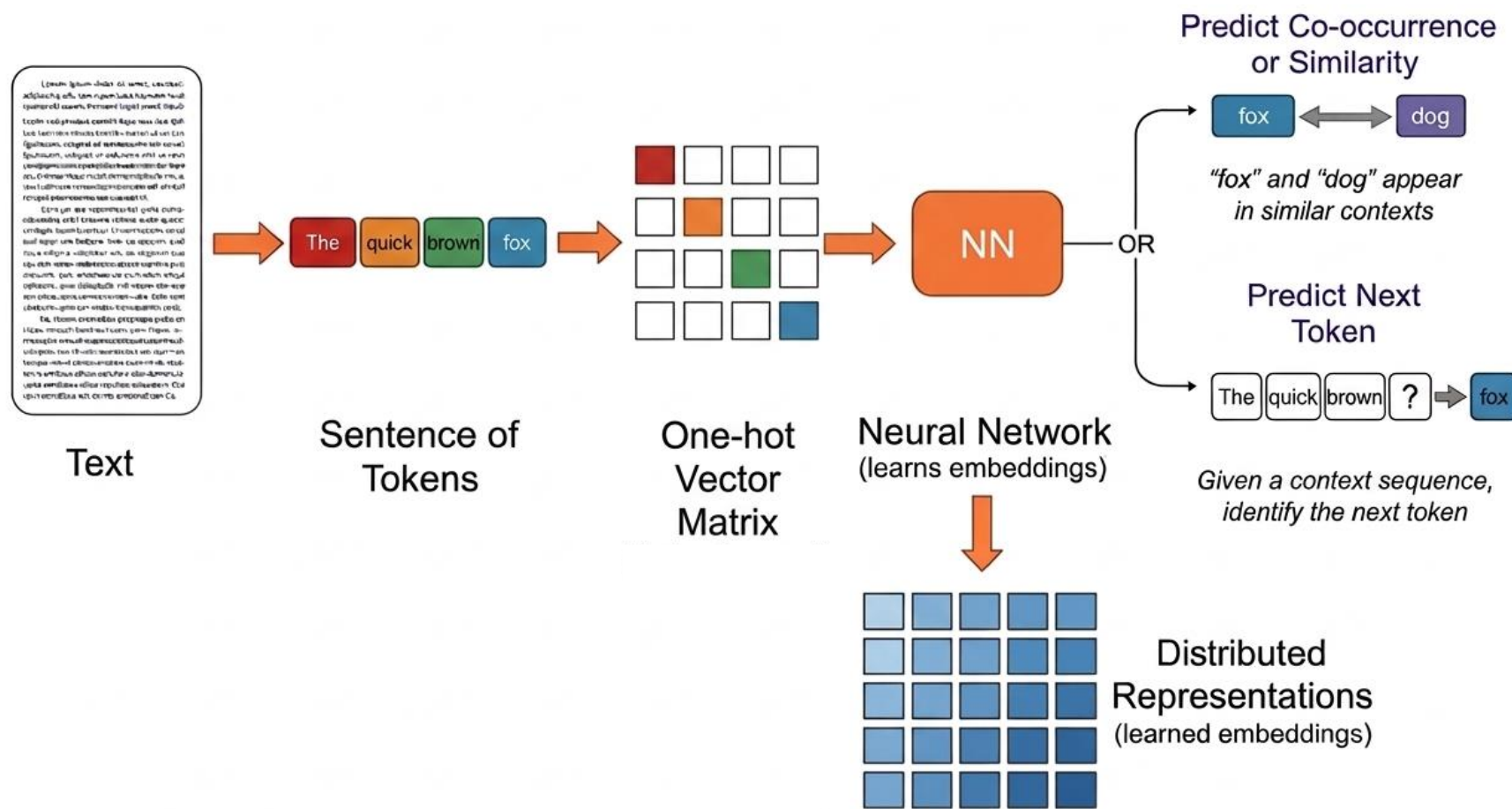
Learn textual representations

- Let's use NN, since they excel in learning patterns from data!
 - i.e., let's use NN to approximate mapping function between local-representation to distributed-representation.
- Let's follow the *distributional hypothesis*¹ as assumption.

words occurring in similar contexts tend to have similar meanings
- I.e. the main idea is to represent words that appear in similar contexts with similar vectors since these words will have similar meanings.

Learn textual representations - Algorithm

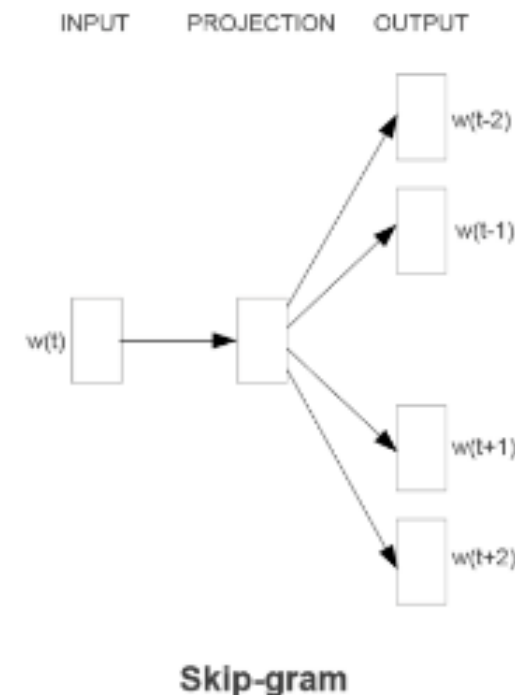
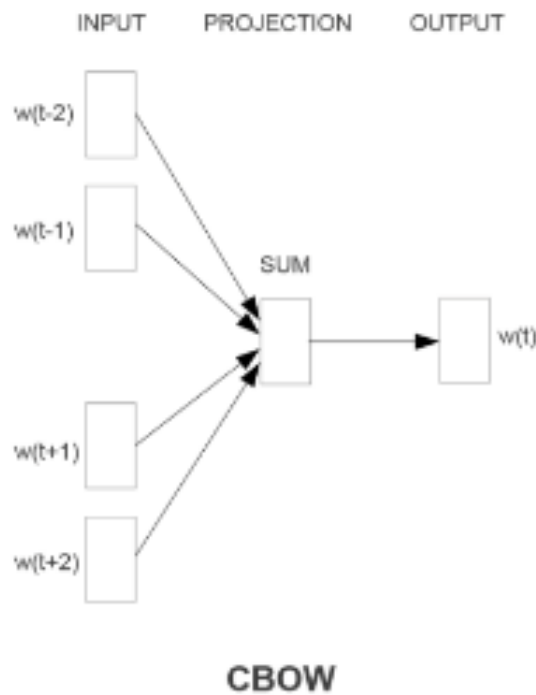
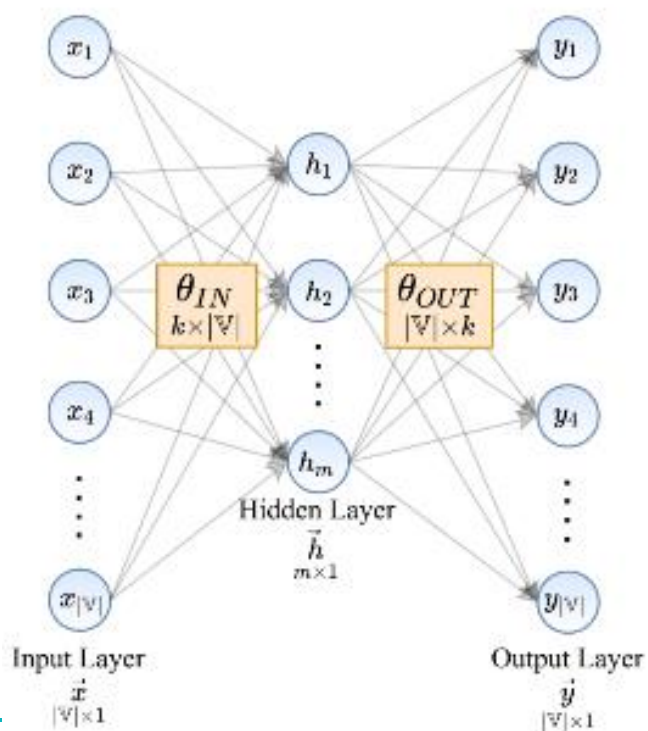
- There are a lot of training objectives, but the pipeline boils down to the following steps.



Learn textual representations

Co-occurrences

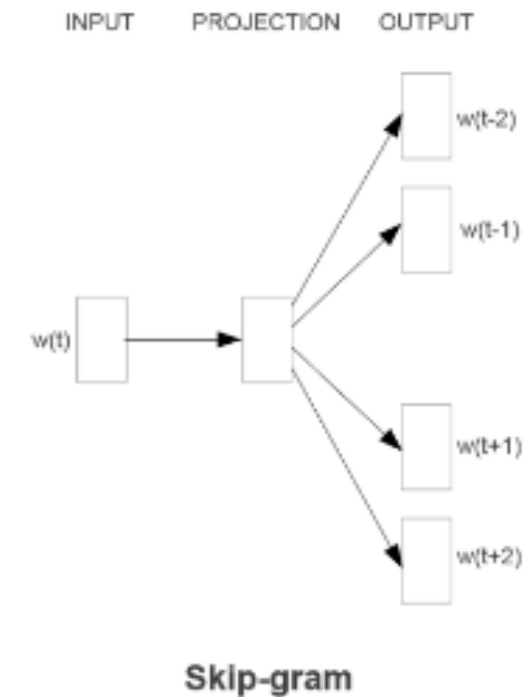
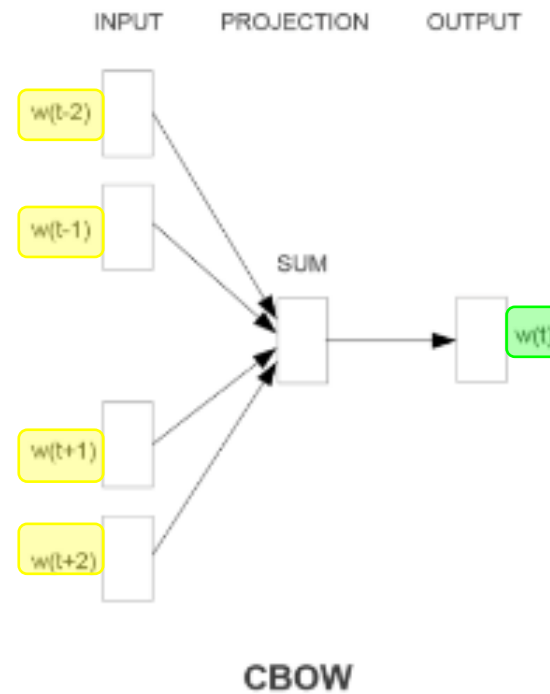
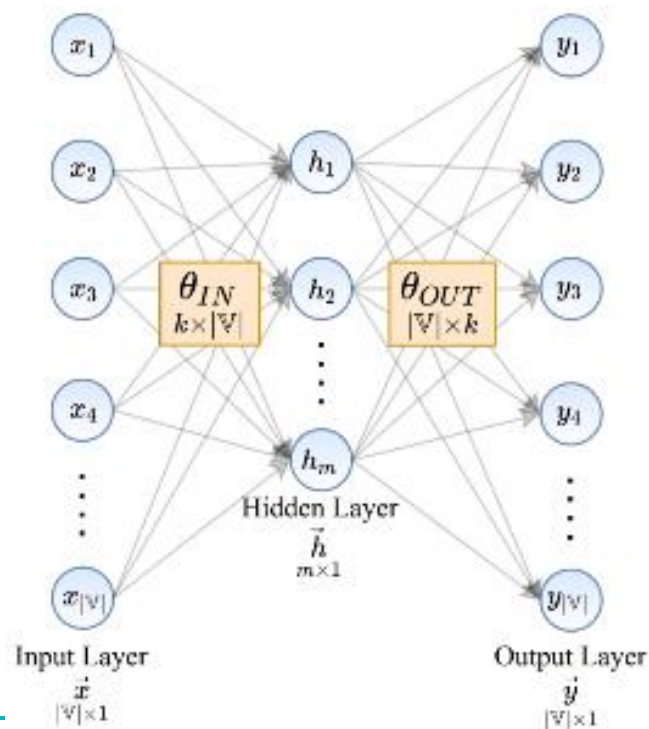
- Word2Vec is an algorithm that follows *distributional* hypothesis train NN to learn distributed representations over large text collections.



Learn textual representations

Co-occurrences

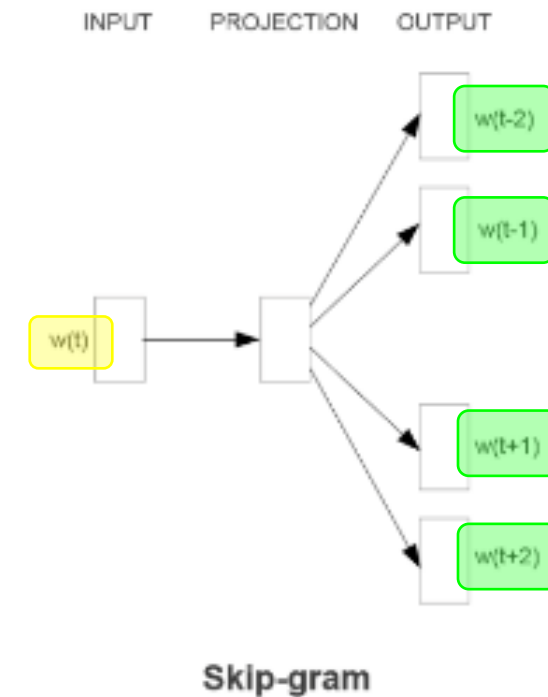
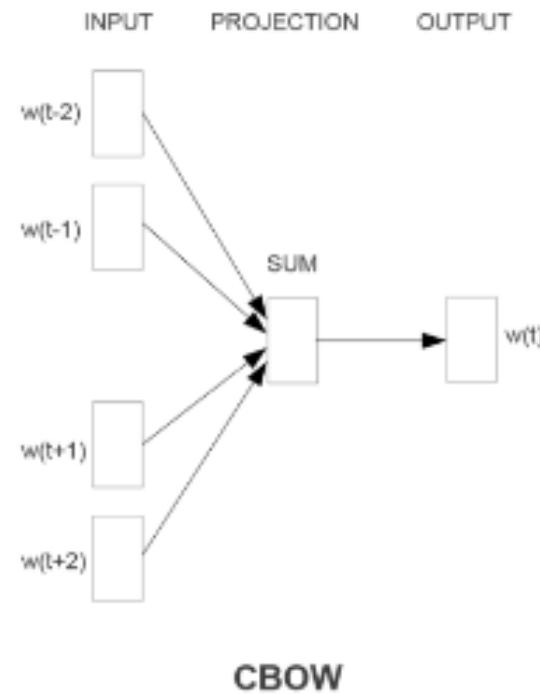
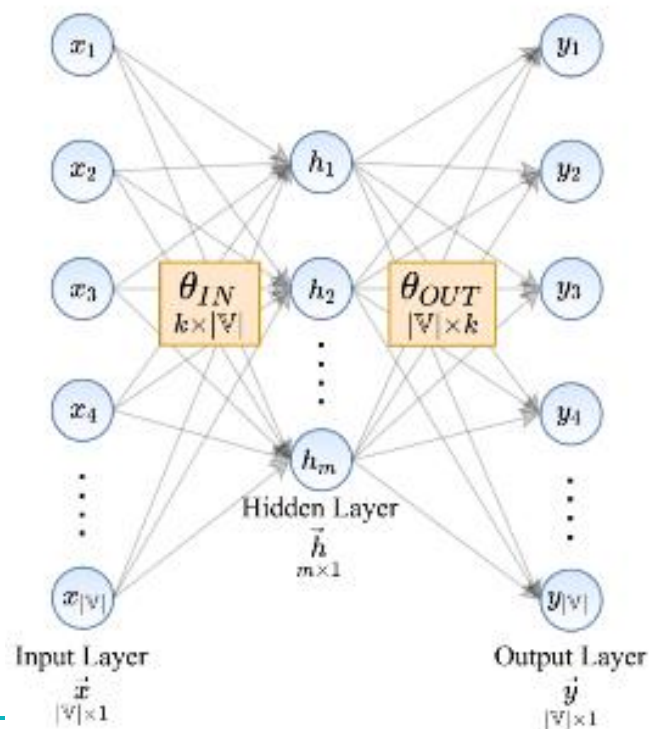
- Word2Vec is an algorithm that follows *distributional* hypothesis train NN to learn distributed representations over large text collections.



Learn textual representations

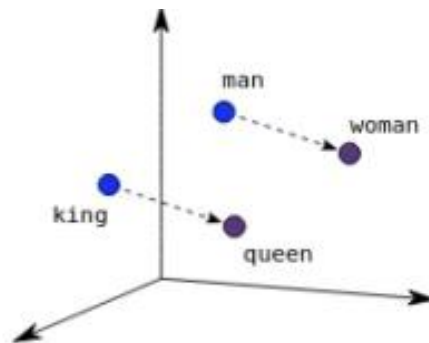
Co-occurrences

- Word2Vec is an algorithm that follows *distributional* hypothesis train NN to learn **distributed representations over large text** collections.

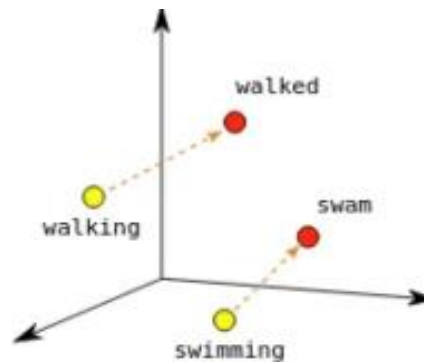


Textual representations - Properties

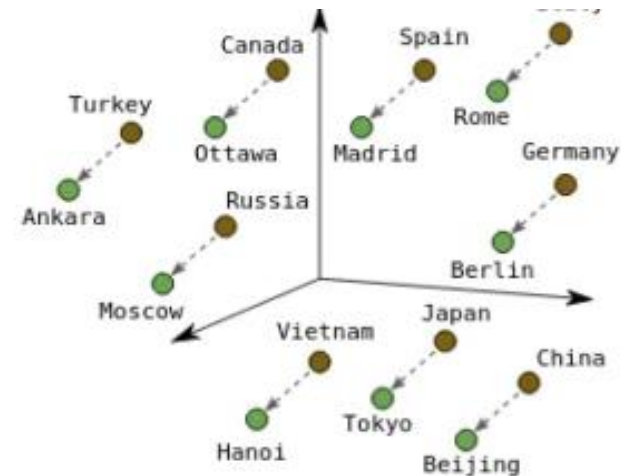
➤ Arithmetic holds



Male-Female



Verb Tense



Country-Capital

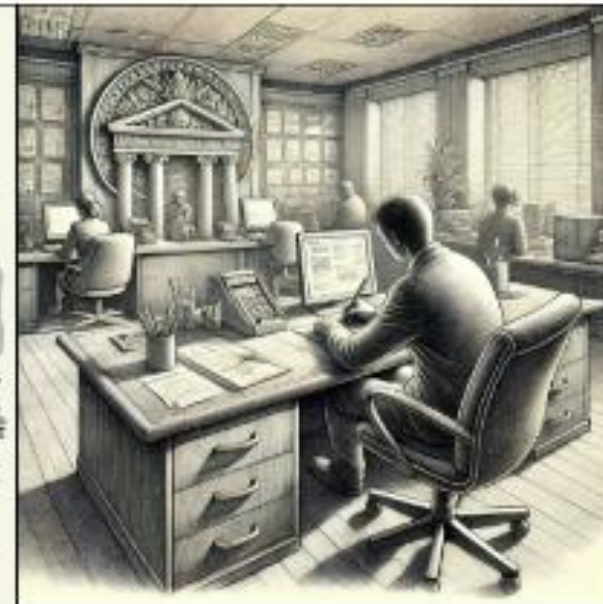
Textual Representation – Word2Vec limitations

- The word representations are static.
 - Each word has a unique fixed representation regardless of its context.
- How it works for polysemous words?
 - Consider the word “bank” in both contexts:

I am sited on the river bank



I am opening a bank account



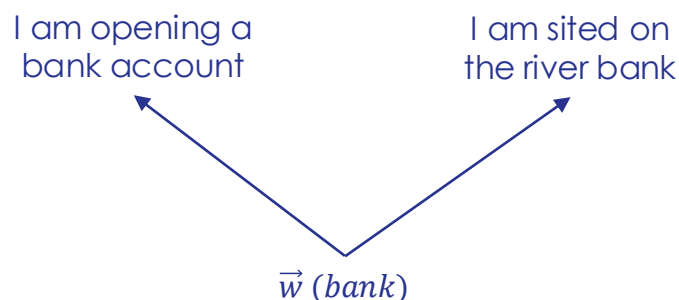
Contextualized representations

- To address this issue, the distributed word representations need to be contextualized by their surrounding words.

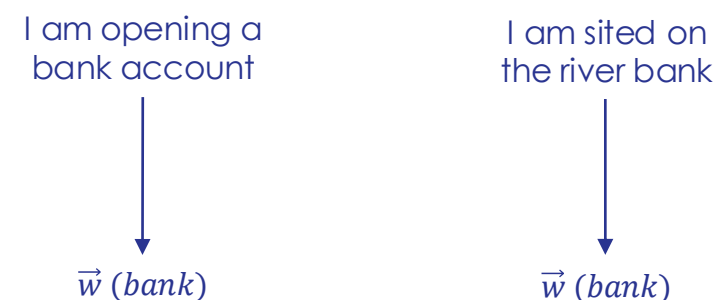


- Creating contextualized distributed representations that are dynamic and change depending on the context.

Static word embeddings

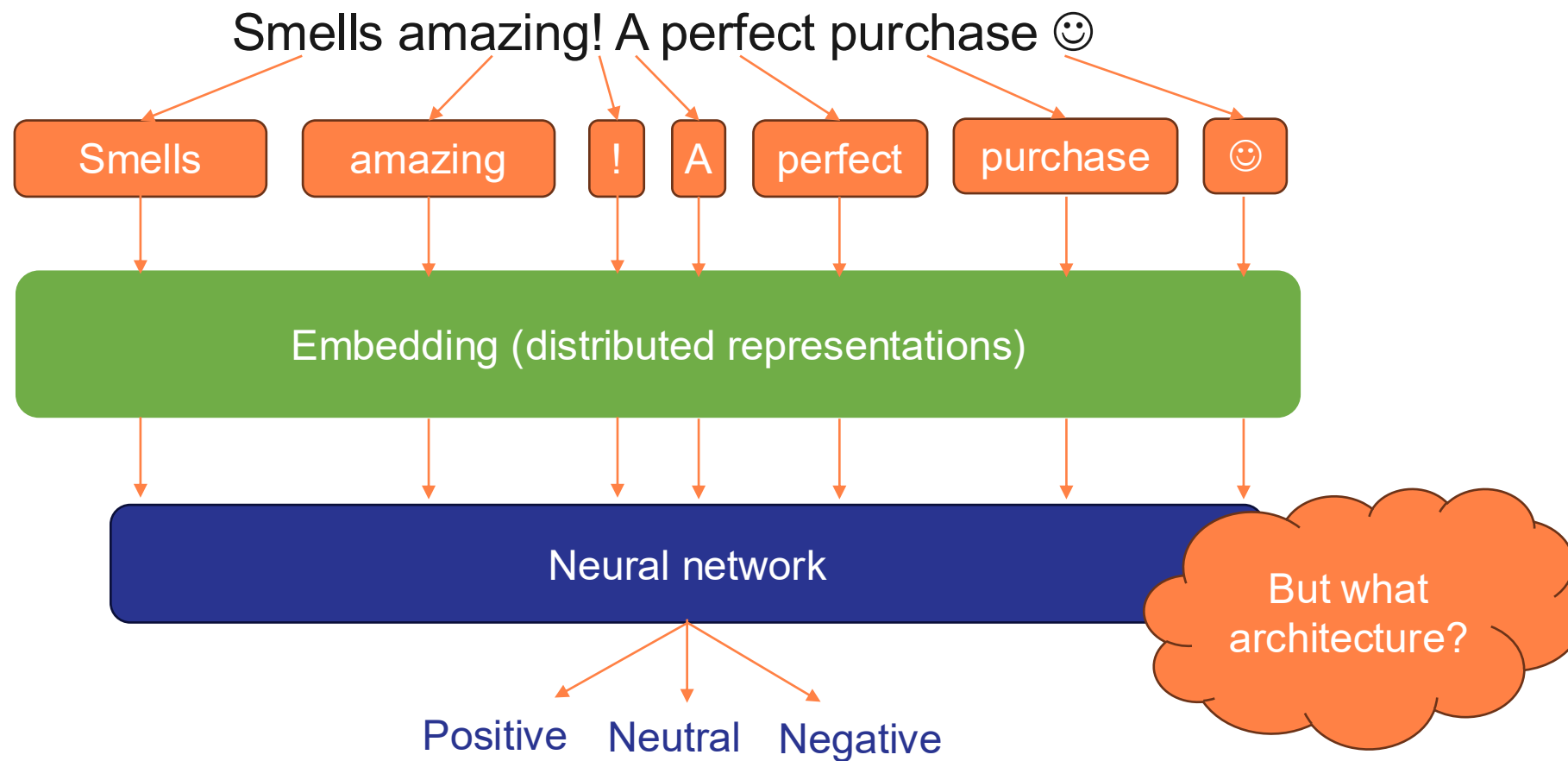


Contextualized word embeddings



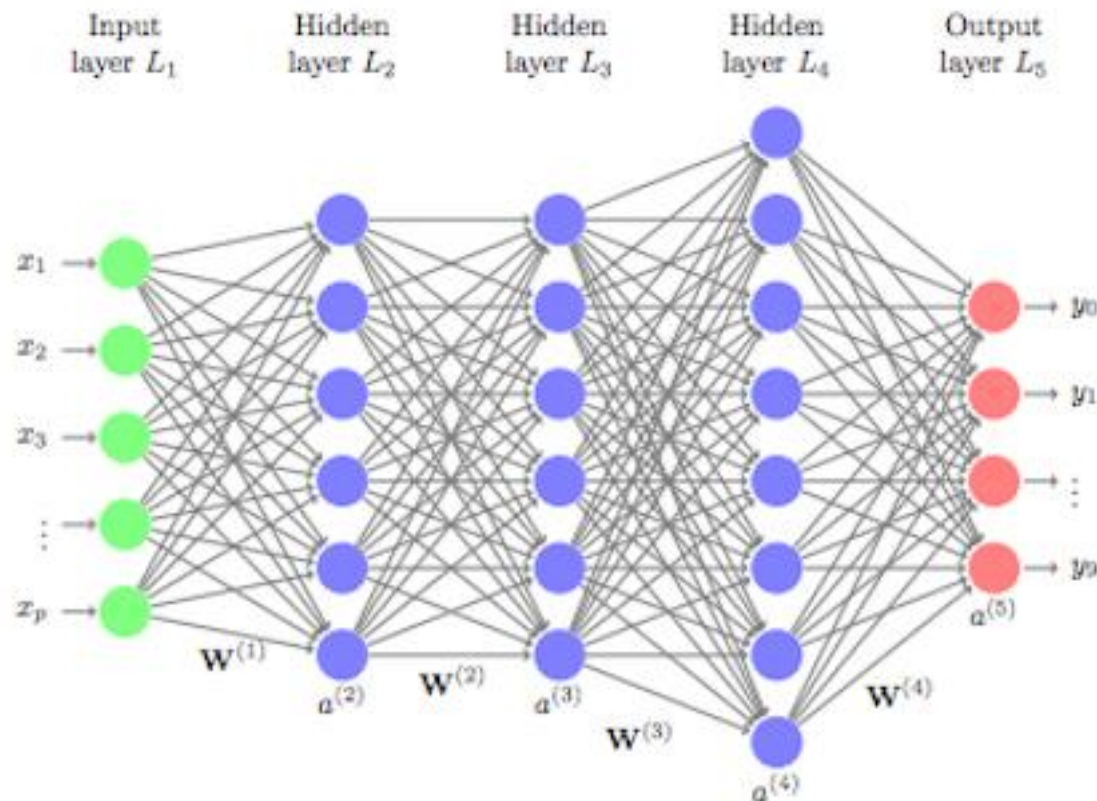
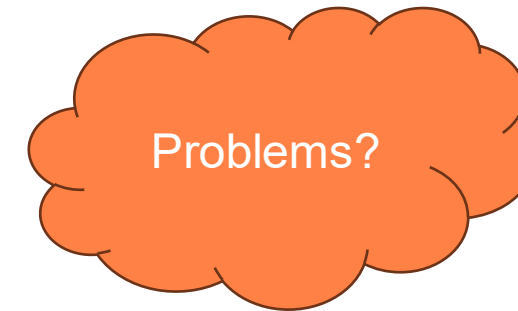
2. How to use these representations

Back to our example



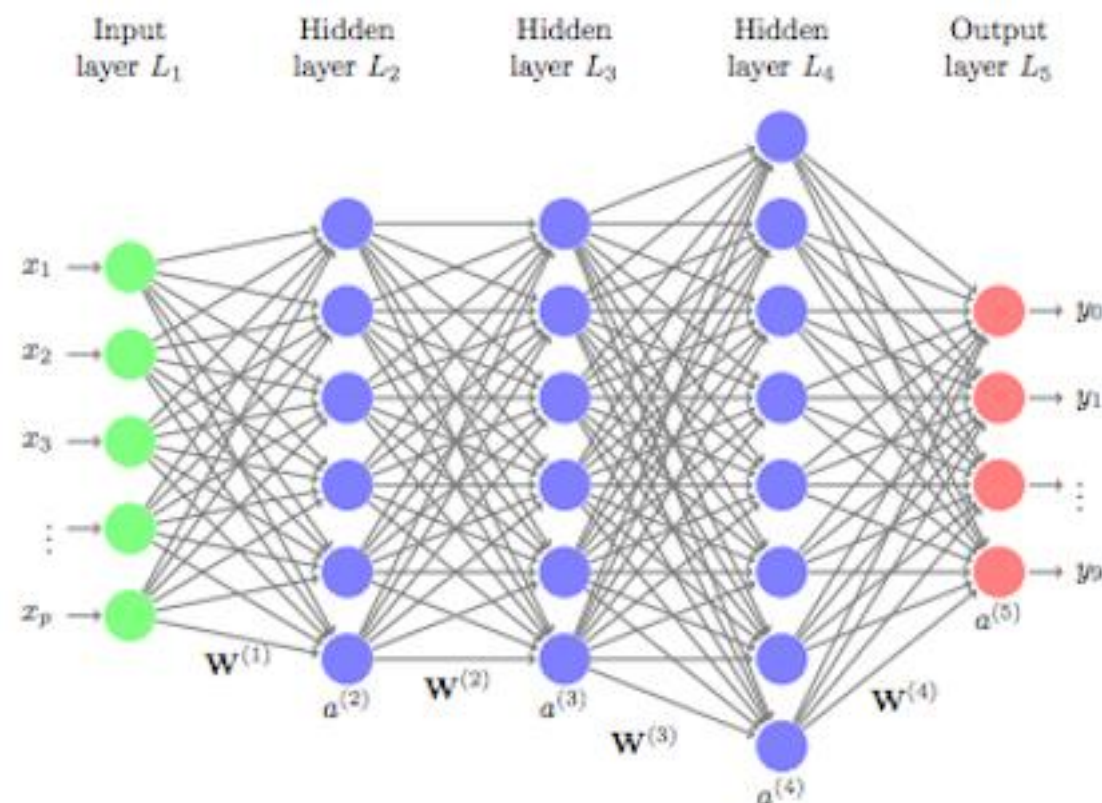
How to process text in NN

- Feed-forward neural network?



How to process text in NN

➤ Feed-forward neural network?



- How do we encode variable length?
 - NN have a strict fixed limit. Padding is not an option
- How do they encode position?
 - Sequence dependency is a property super important in text.

How to process text in NN

➤ 1D - convolution neural networks?

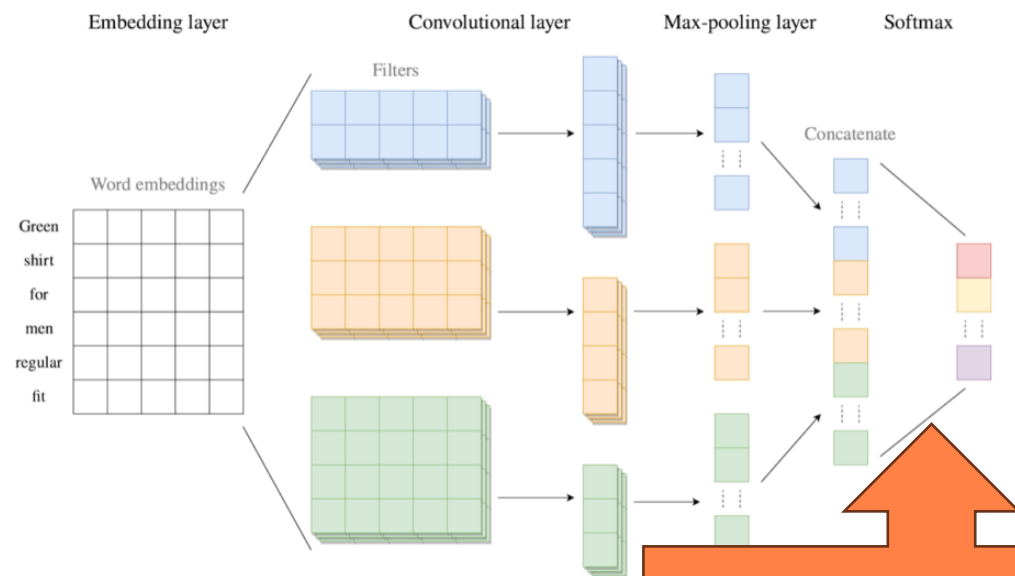


Figure 3: General CNN architecture for text classification

Global max pooling over the time dimension

How to process text in NN

➤ 1D - convolution neural networks?

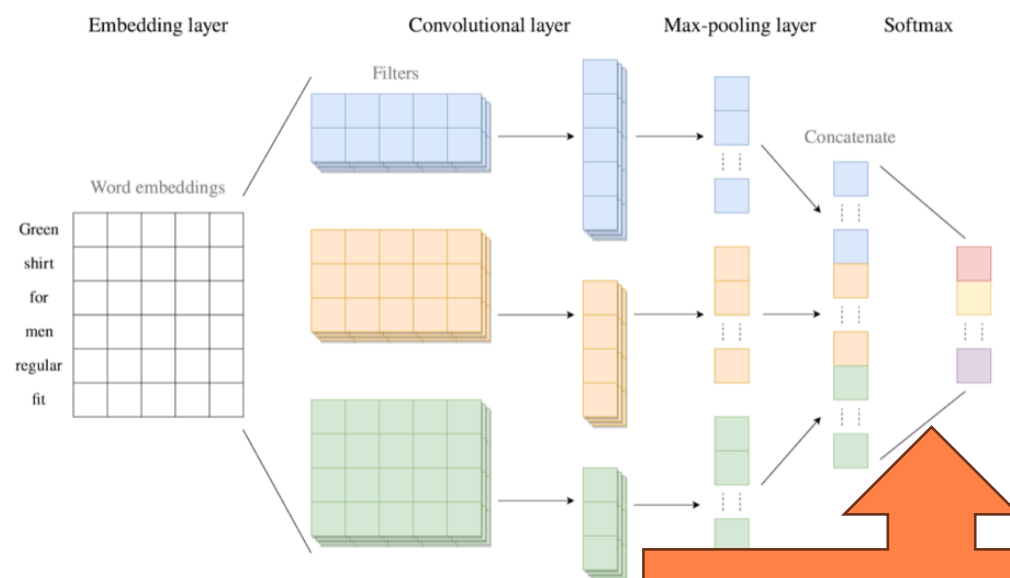


Figure 3: General CNN architecture for text classification



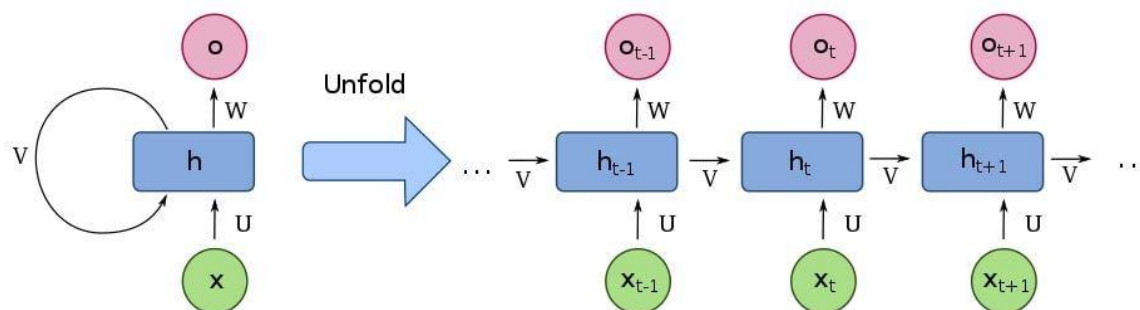
- How do we encode variable length?
 - CNN is natural more fit to handle variable length (we just move our kernel more times)
- How do they encode position?
 - Only locally, not globally because **global** max pooling throw away positional information.

RNN in practice

➤ Recurrent Neural Networks?

➤ Both length and position are encoded as part of the RNN sequence.

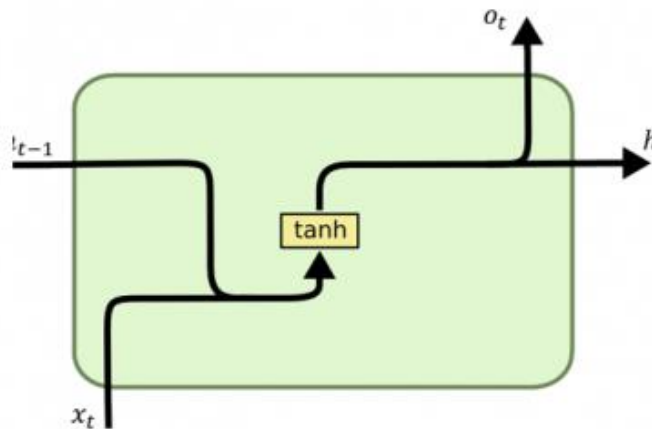
$$h_t = f_\phi(h_{t-1}, x_t)$$



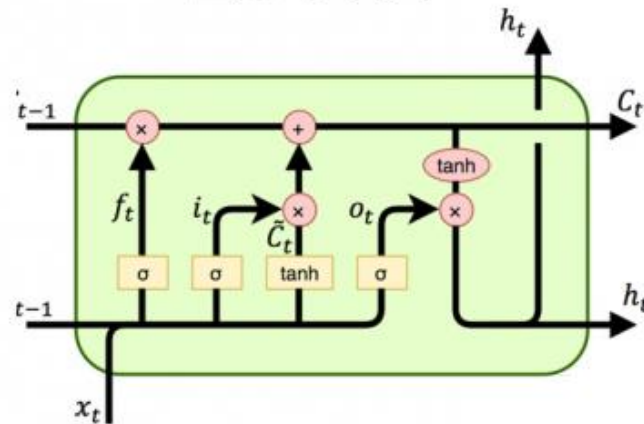
Key idea: RNNs have an “internal state” that is updated as a sequence is processed

Types of RNN

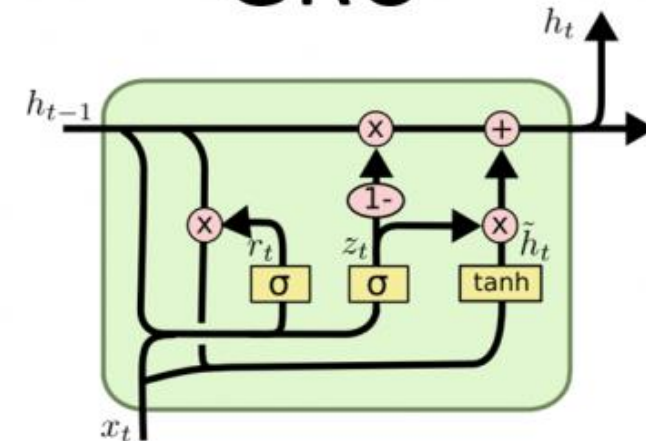
RNN



LSTM



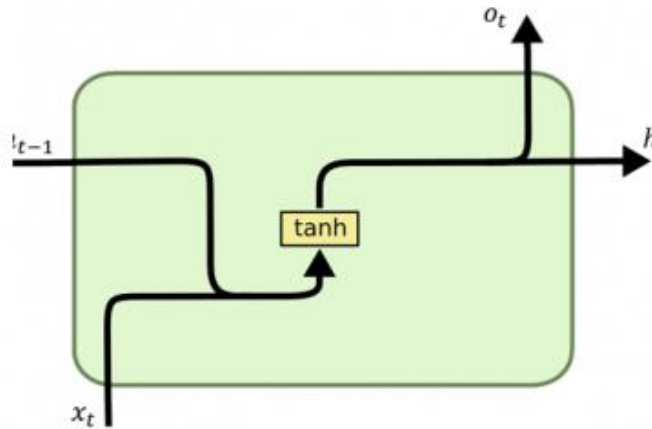
GRU



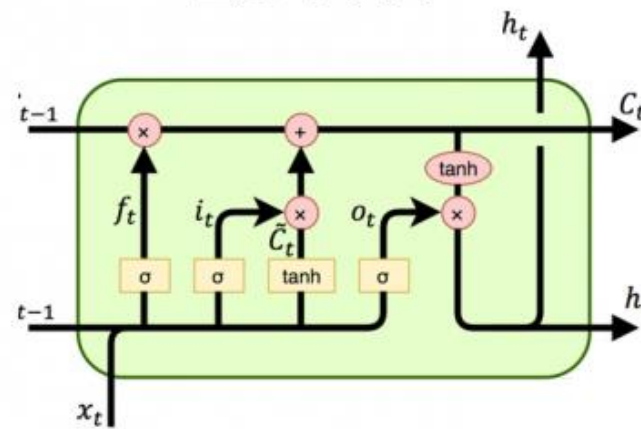
Types of RNN

What is the main disadvantage of an RNN?

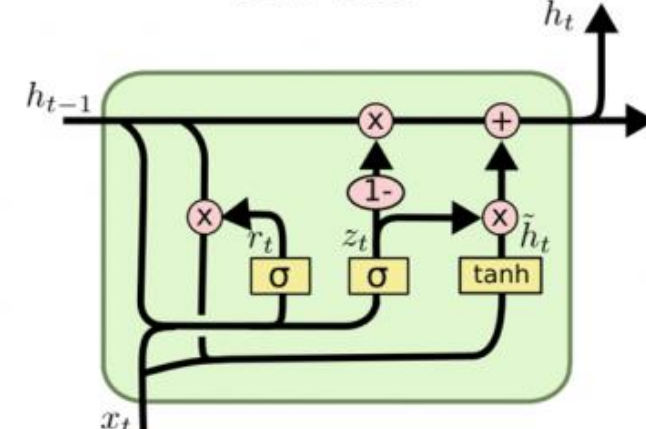
RNN



LSTM

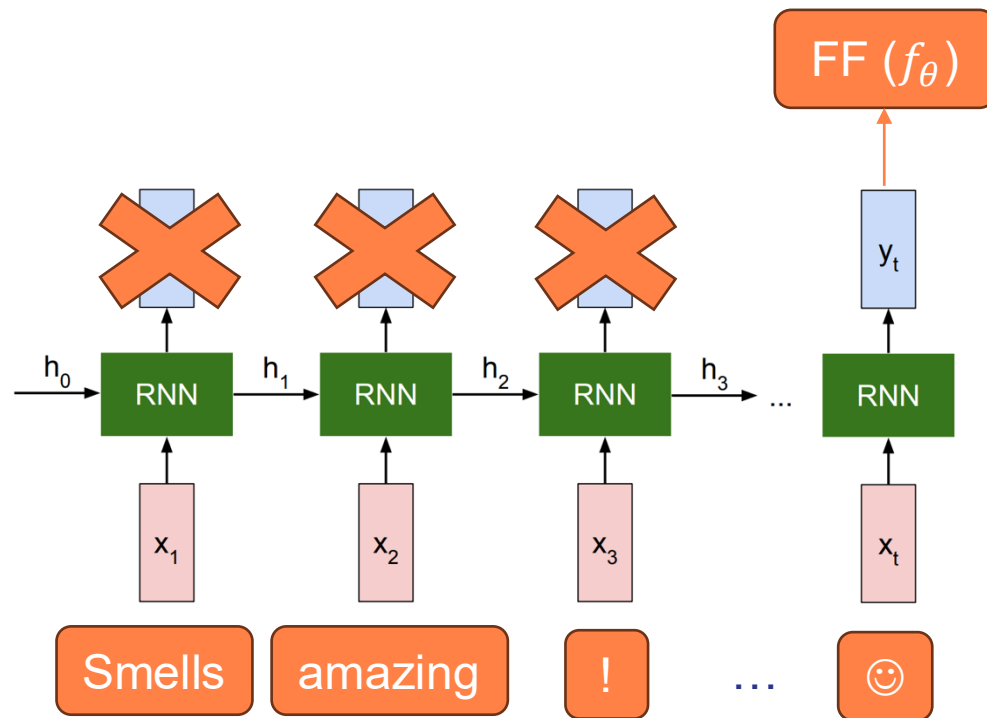


GRU



RNN in practice

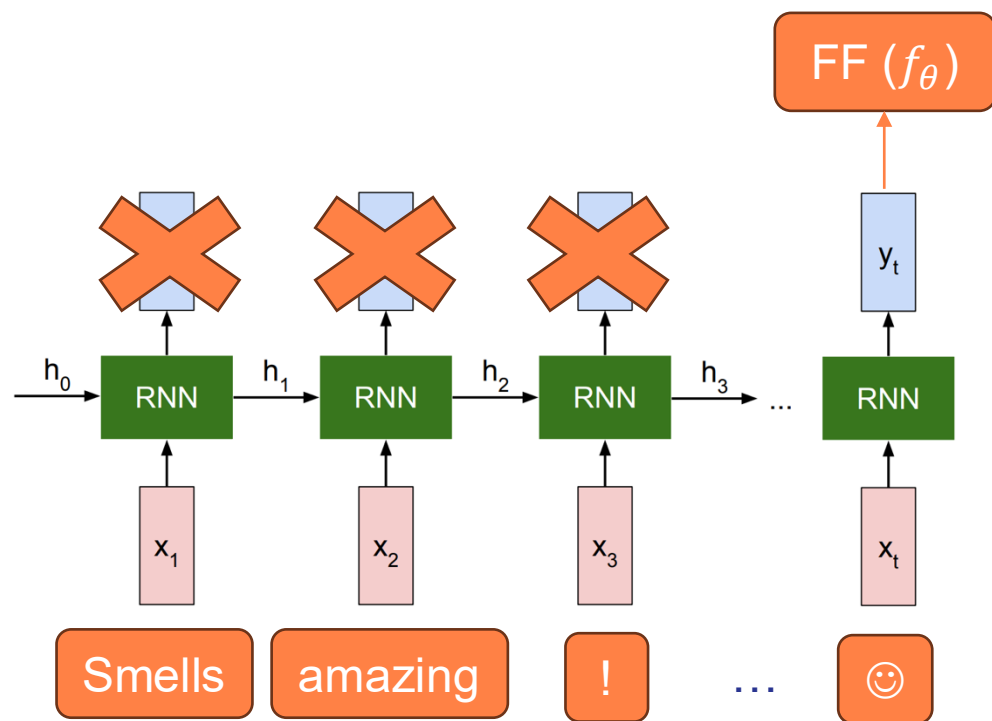
- Practical example for sentiment analysis



$$h_t = f_\phi(h_{t-1}, x_t)$$

$$P(y|x_1, x_2, x_3, \dots, x_t) = f_\theta(h_t)$$

RNN in practice



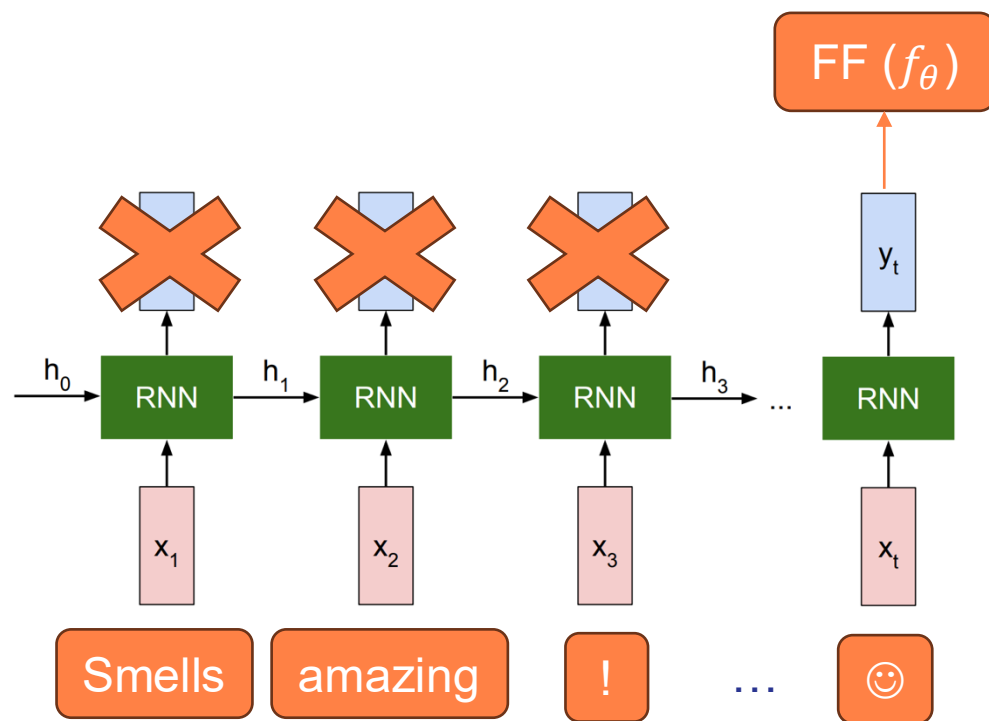
$$h_t = f_\phi(h_{t-1}, x_t)$$

$$P(y|x_1, x_2, x_3, \dots, x_t) = f_\theta(h_t)$$

What if y corresponds to our vocabulary?

RNN in practice

- Can also be used for text generation



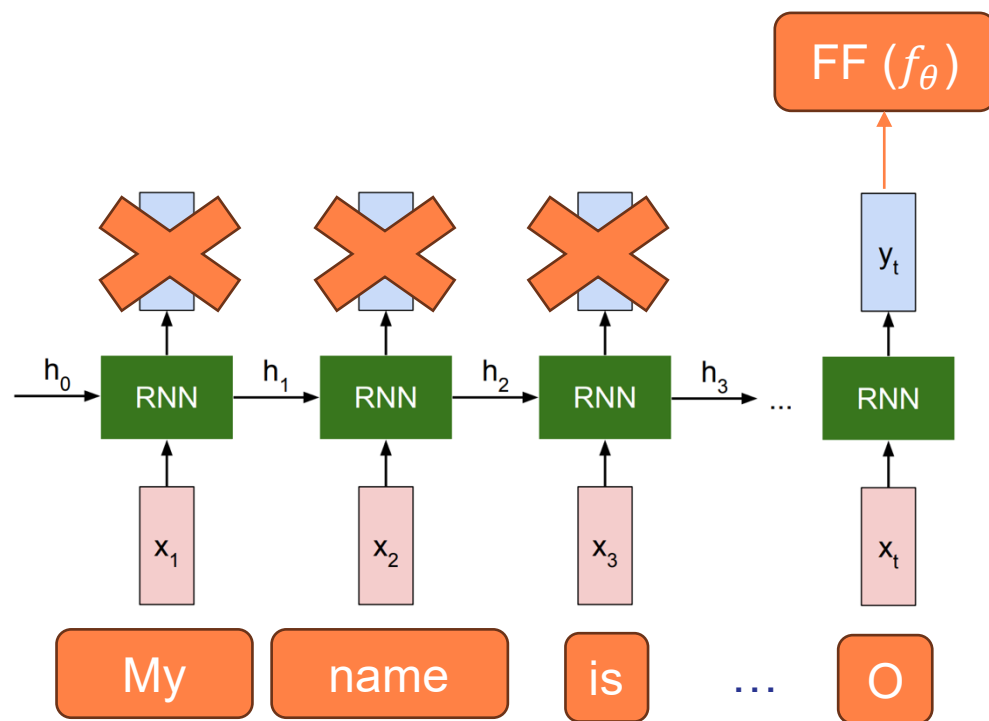
$$h_t = f_\phi(h_{t-1}, x_t)$$

$$P(y|x_1, x_2, x_3, \dots, x_t) = f_\theta(h_t)$$

$$P(w_0, w_1, \dots, w_m) = \prod_{i=0}^m P(w_i | w_0, w_1, \dots, w_{i-1})$$

Special case of translation

- What about a translation task:
 - Goal is given a sentence on a language O, produce a target sentence on a language T, such that the meaning remains the same.



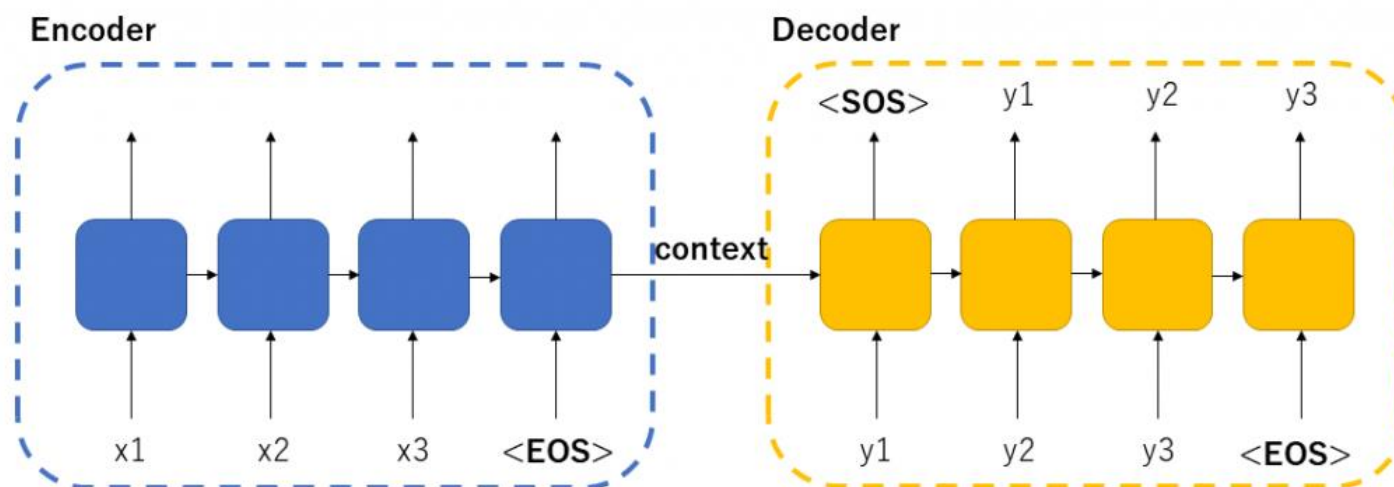
$$h_t = f_\phi(h_{t-1}, x_t)$$

$$P(y|x_1, x_2, x_3, \dots, x_t) = f_\theta(h_t)$$

What is the problem with this approach?

Special case of translation

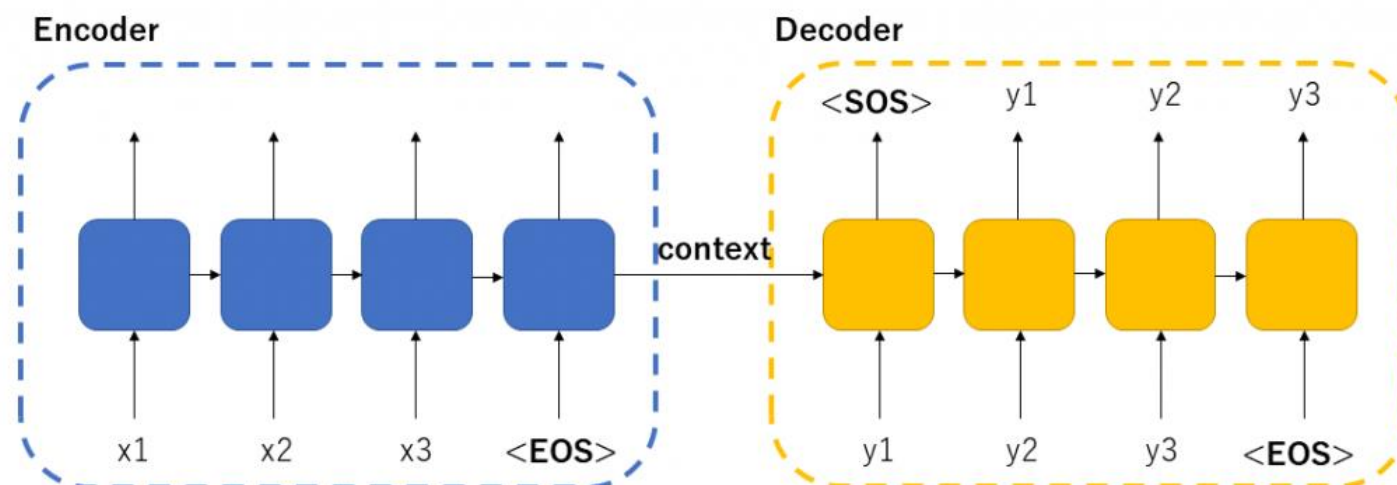
- It assumes that the same network needs to know both languages and needs to be able to predict tokens to both languages.
- Better approach is to separate the tasks of "reading" the input and "writing" the output.
 - Instead of one network doing everything, we use **two separate RNNs**.
 - This leads us to the **Encoder-Decoder (Sequence-to-Sequence)** architecture.



Intuition the first RNN encodes the input sentence into a fixed-vector representation. Second RNN learns to decode it into a target language.

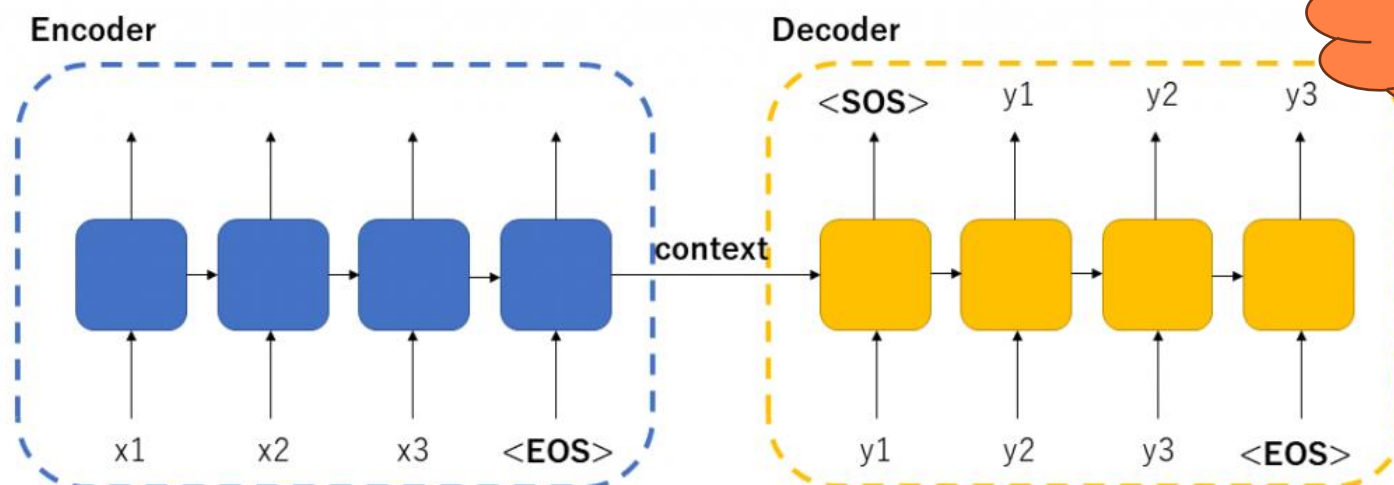
Special case of translation

- This architecture works well for translation, but it is not limited to it, it as being also used for answer generation and other text generation tasks.
- However, it usually only performs well on short spans of text, why?



Special case of translation

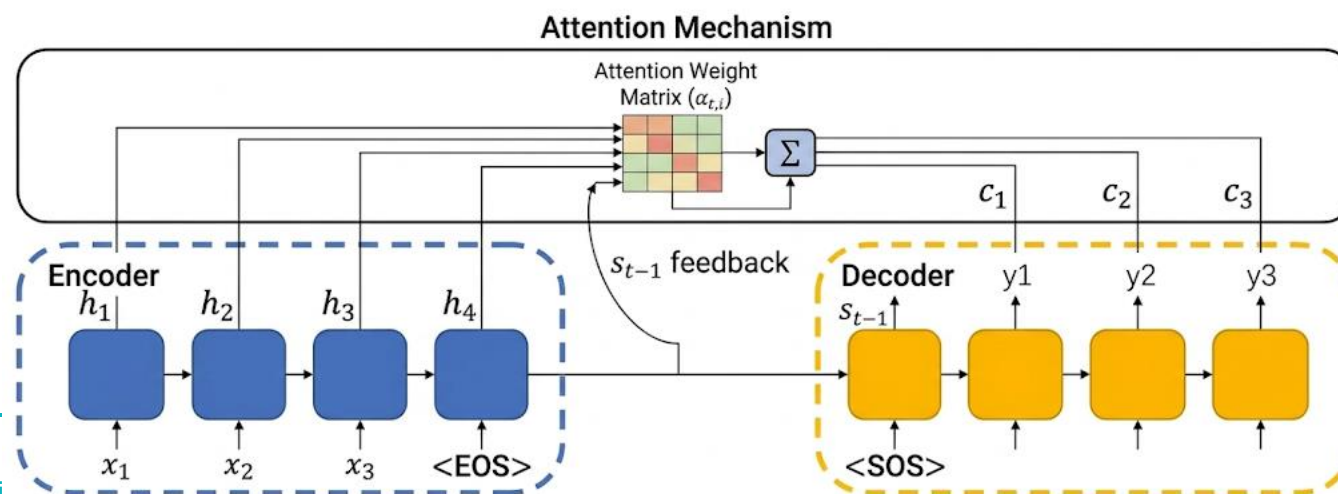
- This architecture works well for translation, but it is not limited to it, it as being also used for answer generation and other text generation tasks.
- However, it usually only performs well on short spans of text, why?
 - The Encoder must compress the entire input sentence—no matter how long—into a single, fixed-size vector



How can we improve on this?

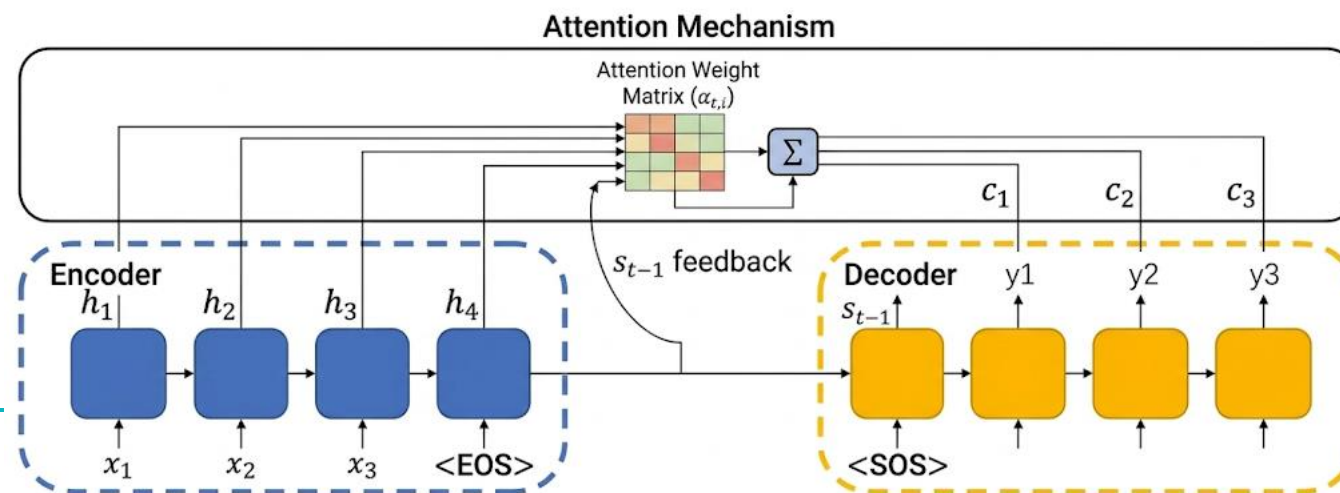
Attention mechanism

- Instead of passing only the single Context Vector, the Encoder also passes **all** of its hidden states to the Decoder.
 - Idea let the decoder see the input sentence, while context vector only cares about the meaning.
- The Decoder learns an **Attention weights** function to dynamically decide which input words are most relevant to the word it is currently translating.
 - Intuition, not every input word has the same important at each timestep, so let the decoder figure out which words it should attend to at each timestep.



How to compute the attention weights

- $c_t = \sum_{i=1}^T a_{t,i} \times h_i$, where T is the total number of tokens in the input sequence.
 - c_t is just average of the input hidden representation weighted by the attention weights.
- $a_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^T \exp(t,j)}$
 - Attention weights corresponds to the normalized scores of the align function, $e_{t,i}$.
- $e_{t,i} = \text{align}(s_{t-1}, h_i)$
 - Computes how well the inputs around position i relate to the output at position t .



Modern attention: Queries, Keys and Values

- To generalize how attention works, modern deep learning uses a database retrieval analogy: **Queries**, **Keys**, and **Values**.
- **Query (Q)**: What the network is currently looking for.
 - In our Seq2Seq model: The decoder's previous hidden state (s_{t-1})
- **Key (K)**: The "labels" of the information we can attend to.
 - In our Seq2Seq model: The encoder's hidden states (h_i).
- **Value (V)**: The actual content we extract once we know the scores.
 - In our Seq2Seq model: Also the encoder's hidden states (h_i). (K=V)

Different types of align function

Alignment model	Equation	Authors
Additive	$align(\vec{q}, \vec{k}) = \vec{v}_{1 \times a}^T \cdot \tanh \left(W_{a \times 2e} \cdot \underbrace{\left\{ \vec{q}_{e \times 1}; \vec{k}_{e \times 1} \right\}}_{\text{concatenation}} \right)$	Bahdanau <i>et al.</i> , 2015
Dot-product	$align(\vec{q}, \vec{k}) = \vec{q}_{1 \times e}^T \cdot \vec{k}_{e \times 1}$	Luong <i>et al.</i> , 2015
Scaled dot-product	$align(\vec{q}, \vec{k}) = \frac{\vec{q}_{1 \times e}^T \cdot \vec{k}_{e \times 1}}{\sqrt{scalar}}$	Vaswani <i>et al.</i> , 2017

Here k is the input sequence and q is the decoder previous step

Types of attention

- Up to this point, we have only looked at **Cross-Attention**.
 - The *Decoder* (producing output) attends to the *Encoder* (processing input).
 - Queries come from the Decoder; Keys and Values come from the Encoder.
- What happens if we apply this exact same attention mechanism to the input sequence *itself*?

Types of attention

- Up to this point, we have only looked at **Cross-Attention**.
 - The *Decoder* (producing output) attends to the *Encoder* (processing input).
 - Queries come from the Decoder; Keys and Values come from the Encoder.
- What happens if we apply this exact same attention mechanism to the input sequence *itself*?
- We get **Self-Attention**
 - The input sequence pays attention to itself to figure out the context of its own words.

Why do we care?
We will see in the next class, with the
introduction of transformers

Sneak peak on transformers



- Limitations of the RNN

Next class - transformers





15 minutes

Hands-On Lab

Thank You!

See you next week!

Lab submission due in 1 week
